Capítulo 4: Spark MLlib

Contents

- Despliegue de modelos en producción con Spark
- Estimadores y transformadores

Spark MLlib es el módulo de Spark para tareas de

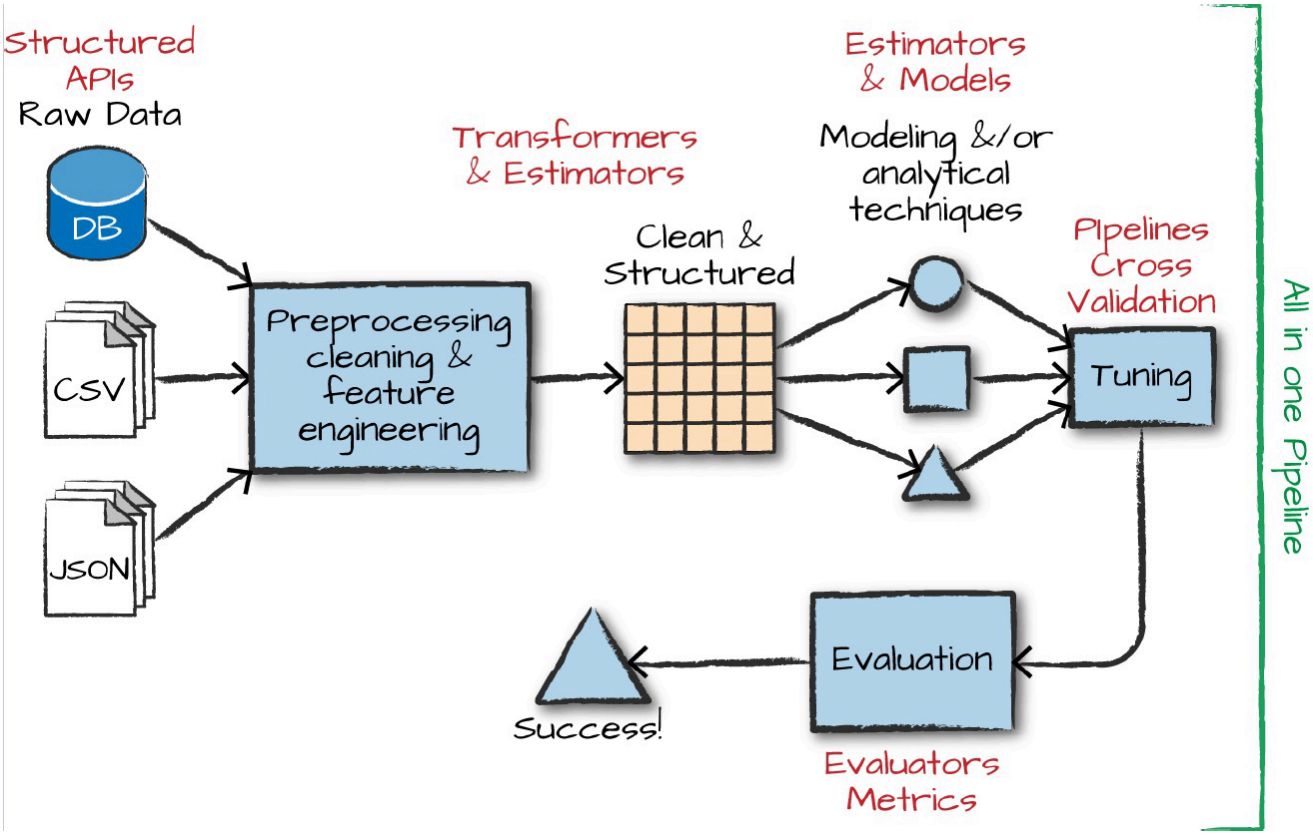
- Limpieza de datos
- Ingeniería de variables (creación de variables desde datos en crudo)
- Aprendizaje de modelos sobre datasets muy grandes (distribuidos)
- Ajuste de parámetros y evaluación de modelos

No proporciona métodos para despliegue en producción de modelos entrenados. En la actualidad, es muy frecuente desplegar modelos como microservicios que usan el modelo entrenado para realizar predicciones sobre un nuevo ejemplo que han recibido por medio de una llamada HTTP de una aplicación externa que les ha enviado el dato para predecir. Comentaremos más detalles en la próxima sección.

La esencia de MLlib es la implementación de modelos de manera distribuida utilizando Spark, y capaces de aprender sobre datasets muy grandes almacenados de manera distribuida. Desde el punto de vista conceptual, los modelos predictivos que ofrece MLlib son los habituales de una biblioteca estándar de Machine Learning, tanto de aprendizaje supervisado como no supervisado. La diferencia con respecto a otros paquetes que también implementan estos métodos, como por ejemplo scikit-learn en Python o caret en R, es que **la implementación se ha realizado desde 0 utilizando operaciones distribuidas** (en especial, operaciones de manipulación de un DataFrame de Spark, las cuales son siempre distribuidas). Como resultado, obtenemos una implementación distribuida de ese algoritmo abstracto. Por supuesto, una implementación que se hizo no-distribuida, como por ejemplo la regresión logística de scikit-learn que en cada paso de la implementación utiliza operaciones no distribuidas, no podrá nunca convertirse automáticamente en distribuida a pesar de ejecutarla en un cluster, puesto que seguirá siendo ejecutada por el intérprete de python que corre en una única máquina, no importa si esa máquina es nuestro PC portátil o es uno de los nodos de un cluster.

Skip to main content

No obstante, también es muy frecuente utilizar solo ciertas funcionalidades de MLlib para pre-procesar variables en datasets masivos, limpiar, normalizar, etc y finalmente filtrar y pasar el dataset resultante (asumiendo que ya no es masivo) al driver para guardarlo en el sistema de archivos local. Posteriormente se puede utilizar una biblioteca de Machine Learning no distribuida, como por ejemplo Scikit-learn de Python o paquetes específicos de R como caret o e1071. Tanto Python como R son capaces de leer ficheros no distribuidos tanto en formato texto como CSV entre otros, y usarlos como entrada para un algoritmo de aprendizaje automático.



La figura anterior muestra una idea general del ciclo de ajuste de un modelo, junto a las herramientas que proporciona Spark para cada etapa (en color rojo). Además, Spark tiene una herramienta adicional llamada pipelines para encapsular todas estas etapas en un solo objeto indivisible, y asegurarnos de que los nuevos datos recibidos y con los que queremos realizar predicciones pasen por exactamente las mismas etapas de pre-procesamiento que el dataset estático con el que se entrenó el modelo. Por eso la figura se refiere a estos pasos como un all-in-one Pipeline.

La etapa de pre-procesamiento incluye, además de las operaciones típicas de un proyecto de Data Science, cierto procesamiento específico de Spark para pasar los datos al formato adecuado de columnas que esperan recibir las implementaciones de cada algoritmo en la API de Spark ML. MLlib incorpora herramientas para esto también.

Skip to main content

Despliegue de modelos en producción con Spark

Spark no fue concebido para explotación online de modelos entrenados, es decir, dar respuestas rápidas (predicciones) a un ejemplo nuevo que se recibe, por ejemplo, desde un sitio web. Por el contrario, la fortaleza de Spark está en entrenar modelos en modo batch (offline) con datasets de gran cantidad de datos.

Aun así, hay varias formas de aprovechar el modelo entrenado obtenido por Spark:

- Entrenar en batch con datos offline almacenados en un sistema de almacenamiento de ficheros como ADLS (probablemente el proceso de creación de variables + entrenamiento llevará un tiempo), y usar el modelo entrenado para predecir (también en modo batch) sobre otro conjunto de datos nuevos preparados para la predicción (esta etapa es mucho más rápida).
  - Es un enfoque muy frecuente en casos de uso reales. Es posible cuando los datos que hay que predecir se conocen al completo porque constituyen una cartera más o menos fija (de clientes, productos, etc) que se actualiza en momentos concretos y a partir de ese momento, debemos calcular una nueva predicción para todos ellos.
  - Las predicciones pre-calculadas se almacenan en ADLS u otro sistema de ficheros, o incluso en una Base de Datos para que sea muy rápido servirlas desde un microservicio si fuese necesario servirlas de esa manera.
- Entrenar, guardar el modelo entrenado, y usarlo desde Spark para hacer predicciones. Poco recomendable: lanzar un job nuevo para cada ejemplo que predecir implica sobrecarga. Balanceo de carga con réplicas del modelo.
- Entrenar y exportar el modelo a un formato de intercambio, ej. PMML, para leerlo y explotarlo con otra herramienta no distribuida (Python en especial aunque también R soporta PMML).
- Entrenar en modo online usando Structured Streaming para recoger datos. Siempre re-entrenar desde 0 salvo si el modelo no está preparado para esto (modelos que soporten online learning, que en la actualidad son una minoría).

Estimadores y transformadores

Antes de describir en detalle la API del módulo Spark MLlib, hay que tener en cuenta que en la API de Spark (Java/Scala/Python) se distinguen:

Skip to main content

- Paquete org.apache.spark.mllib (en Python: pyspark.mllib): API antigua basada en RDDs de una estructura llamada LabeledPoint: `LabeledPoint(etiqueta, [vector de atributos])`. Obsoleta, no debe usarse.
- Paquete org.apache.spark.ml (en Python: pyspark.ml): API actual, sobre DataFrames. Se debe utilizar siempre.

Para la creación de variables y el preprocesamiento, en general se utiliza la API de Spark SQL que vimos en el tema anterior. No obstante, Spark ML ofrece una transformación en la que puede escribirse código SQL arbitrario e incluir dicha transformación en un pipeline. Además, ciertas transformaciones relacionadas con el pre-procesamiento estadístico de datos (normalización, estandarización, codificación one-hot, etc) también están disponibles en Spark ML. Por último, existen varias transformaciones cuyo propósito no es realmente modificar los datos sino prepararlos (dar a las columnas del DataFrame los tipos adecuados) para la entrada de los algoritmos de Machine Learning de Spark. En concreto, Spark requiere estos formatos:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()

classification_df = spark.createDataFrame([    # Ejemplo de DF para problemas de cla
    ([-32.2, 4.5, 1.0, 6.7], 1.0),             # con números reales con la parte dec
    ([-40.8, 2.25, 4.0, 2.3], 0.0)             # Ejemplos: 0.0, 1.0, 2.0 ...
], ["features", "class"])

regression_df = spark.createDataFrame([        # Ejemplo de DF para problemas de reg
    ([-32.2, 4.5, 1.0, 6.7], 2.87),
    ([-40.8, 2.25, 4.0, 2.3], -5.41)
], ["features", "target"])

clustering_df = spark.createDataFrame([        # Ejemplo de DF para problemas de clu
    ([-32.2, 4.5, 1.0, 6.7],),
    ([-40.8, 2.25, 4.0, 2.3],)
], ["features"])

recommendation_df = spark.createDataFrame([        # Ejemplo de DF para problemas de
    (2, 10, 4.5),                                  # con evaluaciones explícitas
    (3, 8, 3.2)
], ["user", "item", "rating"])

classification_df.show(truncate=False)
regression_df.show(truncate=False)
clustering_df.show(truncate=False)
recommendation_df.show(truncate=False)
```

```
+----------------------+-----+
|features              |class|
+----------------------+-----+
```

Skip to main content

```
|[-40.8, 2.25, 4.0, 2.3]|0.0  |
+-----------------------+-----+
```

```
+-----------------------+------+
|features               |target|
+-----------------------+------+
|[-32.2, 4.5, 1.0, 6.7] |2.87  |
|[-40.8, 2.25, 4.0, 2.3]|-5.41 |
+-----------------------+------+
```

```
+-----------------------+
|features               |
+-----------------------+
|[-32.2, 4.5, 1.0, 6.7] |
|[-40.8, 2.25, 4.0, 2.3]|
+-----------------------+
```

```
+----+----+------+
|user|item|rating|
+----+----+------+
|2   |10  |4.5   |
|3   |8   |3.2   |
+----+----+------+
```

Como vemos, los valores de las variables deben presentarse en una sola columna de tipo vector. Por otro lado, si se trata de un problema de aprendizaje supervisado (clasificación o regresión), la columna target debe ser siempre de tipo real (Double). En el caso de los problemas de clasificación, cada clase se indica con un número real empezando en 0.0 y con la parte decimal del número siempre a 0 (es decir, si tenemos un problema con 5 clases, se tienen que codificar como 0.0, 1.0, 2.0, 3.0 y 4.0). En el caso de los problemas de regresión, la columna target puede ser cualquier número real. Los nombres de las columnas no tienen por qué ser features y label como en los ejemplos de arriba, sino que pueden ser cualesquiera, siempre que le indiquemos al algoritmo cómo se llama la columna (de tipo vector) que contiene las features en el DataFrame que le pasamos para entrenar, y cómo se llama la columna target, si la hay.

En el caso de algoritmos de recomendación, Spark solo permite filtrado colaborativo . En ese caso hay que pasarle un DataFrame que contenga, al menos, tres columnas con los identificadores de un usuario, un ítem y el rating que ha dado ese usuario a ese ítem, ya sea implícito o explícito. Los identificadores de usuarios y de ítems tienen que ser códigos de tipo entero, mientras que el rating puede ser un número real en cualquier rango.

Según la documentación oficial de Spark, actualmente nos ofrece los siguientes

Skip to main content

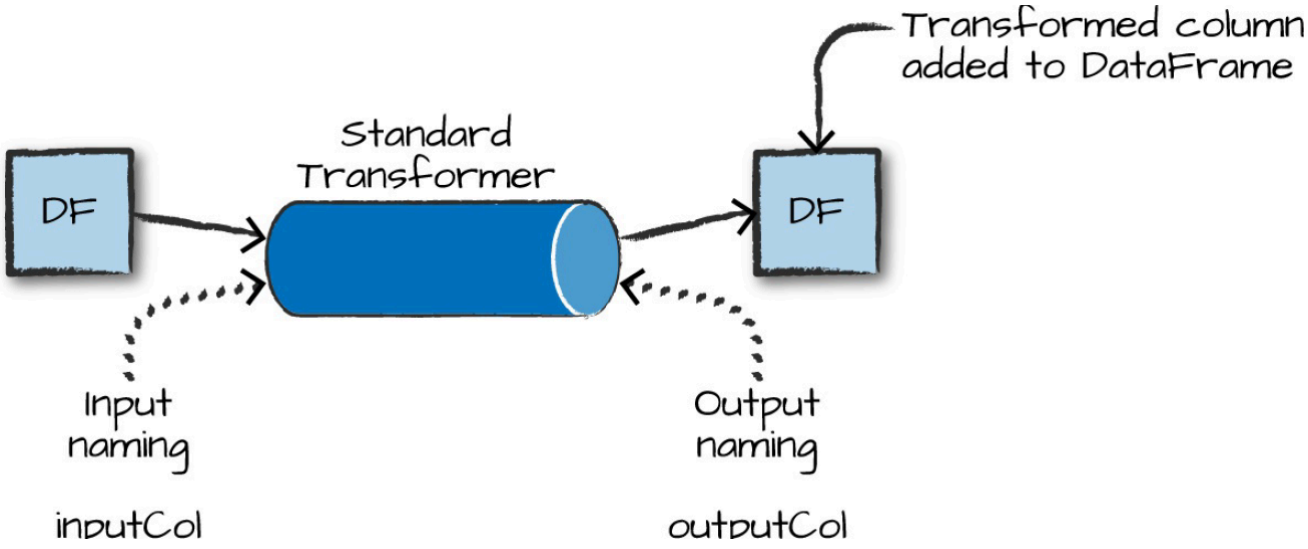
- Extracción: extraer variables a partir de datos en crudo
- Transformación: re-escalar, convertir o modificar variables
- Seleccionar: seleccionar un buen subconjunto de variables de otro más grande
- Locality Sensitive Hashing (LSH): combinación de trasnformaciones de variables con otros algoritmos.

Para más detalles, puede consultarse la página de la documentación de Spark sobre piezas de preprocesamiento en https://spark.apache.org/docs/latest/ml-features.html

Transformadores de Spark ML

Un *transformer* es un objeto que recibe como entrada un DataFrame de Spark y uno o varios nombres de columna existentes (por ejemplo inputCol), y las transforma de alguna manera. Su salida es el mismo DataFrame con una nueva columna añadida a por la derecha, con el nombre que hayamos indicado en el parámetro correspondiente (generalmente outputCol pero a veces puede ser predictionCol).

La interfaz Transformer tiene un único método: transform(df: DataFrame) que recibe un DataFrame y devuelve otro DataFrame. Los transformadores no necesitan aprender ningún parámetro del DataFrame de entrada. Desde el momento de ser creados, ya están preparados (tienen toda la información) para transformar un DataFrame, y esto es lo que hacen cuando llamamos a `transform(df)` tal como muestra la siguiente figura. La única condición es que el DataFrame que le pasamos como argumento contenga la(s) columna(s) que ese transformador tiene configuradas para transformar. No importa qué otras columnas adicionales existan en el DataFrame - eso sí, no puede contener columnas que tengan el mismo nombre que las que el transformador va a añadir nuevas al DataFrame resultante de la transformación.



Skip to main content

Vemos a continuación algunos transformadores frecuentes, aunque la lista de todos los disponibles es mucho más larga, tal como puede consultarse en la documentación de Spark mencionada más arriba para las piezas de preprocesamiento.

VectorAssembler

Recibe varias columnas y las concatena en una sola de tipo vector, de longitud igual al número de columnas que se quieran ensamblar. Necesario para calcular la columna (única) de features en los algoritmos de aprendizaje supervisado. IMPORTANTE: sólo puede ensamblar columnas numéricas o de vectores numéricos, por lo que cualquier columna categórica debe haberse convertido previamente en numérica con `StringIndexer`. Ejemplo:

```
from pyspark.ml.linalg import Vectors
from pyspark.ml.feature import VectorAssembler

dataset = spark.createDataFrame(
    [(0, 18, 1.0, Vectors.dense([0.0, 10.0, 0.5]), 1.0)],
    ["id", "hour", "mobile", "userFeatures", "clicked"])

assembler = VectorAssembler(inputCols=["hour", "mobile", "userFeatures"],
                            outputCol="features")

output = assembler.transform(dataset)

print("DataFrame original:")
dataset.show()

print("DataFrame resultado de ensamblar hour, mobile, userFeatures en la columna fe
output.show(truncate=False)
```

```
DataFrame original:
```

```
+---+----+------+--------------+-------+
| id|hour|mobile|  userFeatures|clicked|
+---+----+------+--------------+-------+
|  0|  18|   1.0|[0.0,10.0,0.5]|    1.0|
+---+----+------+--------------+-------+

DataFrame resultado de ensamblar hour, mobile, userFeatures en la columna features:
```

```
+---+----+------+--------------+-------+-----------------------+
|id |hour|mobile|userFeatures  |clicked|features               |
+---+----+------+--------------+-------+-----------------------+
|0  |18  |1.0   |[0.0,10.0,0.5]|1.0    |[18.0,1.0,0.0,10.0,0.5]|
```

Skip to main content

SQLTransformer

Recibe una sentencia SQL de tipo SELECT-FROM-WHERE, expresada string, y la aplica al DF que se le pase como argumento a transform(). Útil para dejar fijadas series de transformaciones arbitrarias que luego sean incluidas en un pipeline. Hay que recordar que, en general, en un proceso de ML, el código que escribimos para pre-procesar, limpiar, generar nuevas features, etc, no suele estar incluido como tal dentro de una pieza de tipo `Pipeline` (en el sentido que veremos luego, pipeline de Spark ML). Lo que sucede es que el desarrollador es responsable de asegurarse de que el mismo código que hace todo este proceso durante la etapa de entrenamiento (para convertir datos en crudo en features utilizadas por el modelo para entrenar) será también ejecutado en la etapa de predicción: cuando el modelo se despliega y recibe nuevos datos para predecir, los cuales también deben ser pre-procesados y convertidos en las features que el modelo requiere para poder predecir sobre ellas. Dicho código (que debería ser "único", y no estar duplicado para las etapas de entrenamiento y de predicción), generalmente no suele/puede convertirse en una "etapa de pipeline", puesto que ese tipo de etapas suelen tener funcionalidad muy específica y limitada. Aun así, podemos expresar ciertas transformaciones arbitrarias dentro de una sentencia SQL, que será aplicada sobre el DataFrame recibido como argumento por el método `transform()` de dicha pieza.

```
from pyspark.ml.feature import SQLTransformer

df = spark.createDataFrame([
    (0, 1.0, 3.0),
    (2, 2.0, 5.0)
], ["id", "v1", "v2"])

sql_transformer = SQLTransformer(
    statement = "SELECT *, (v1 + v2) AS v3, (v1 * v2) AS v4 FROM __THIS__"
)
result_df = sql_transformer.transform(df)  # añadirá las nuevas columnas v3 y v4
result_df.show()
```

```
+---+---+---+---+----+
| id| v1| v2| v3|  v4|
+---+---+---+---+----+
|  0|1.0|3.0|4.0| 3.0|
|  2|2.0|5.0|7.0|10.0|
+---+---+---+---+----+
```

Skip to main content

Bucketizer

Discretizador que transforma una columna continua en una columna con identificadores de intervalo (bucket). Recibe como parámetro el vector con los límites de cada intervalo. La columna resultante codifica las categorías como números reales con la parte decimal a 0 y empezando en 0.0, es decir, tal como Spark necesita representar columnas categóricas. Además, dicha columna contiene **metadatos** que, aunque no se visualizan al mostrar por pantalla el contenido del DataFrame, son utilizados por los algoritmos para interpretar correctamente esa columna como categórica y actuar en consecuencia al entrenar un modelo. Los puntos de corte indicados se interpretan por parejas como intervalos. Por ejemplo, la lista de puntos de corte [-3.2, 0.6, 3.5, 8] corresponde a los intervalos [-3.2, 0.6), [0.6, 3.5) y [3.5, 8), donde cada intervalo siempre es cerrado por la izquierda (incluye al extremo inferior) y abierto por la derecha (no incluye al extremo superior).

```
from pyspark.ml.feature import Bucketizer

splits = [-float("inf"), -0.5, 0.0, 0.5, float("inf")]
data = [(-999.9,), (-0.5,), (-0.3,), (0.0,), (0.2,), (999.9,)]

dataFrame = spark.createDataFrame(data, ["features"])

bucketizer = Bucketizer(splits = splits, inputCol = "features", outputCol = "bucket

# Transformamos los datos originales en el índice de su intervalo
bucketedData = bucketizer.transform(dataFrame)

# Obtenemos la longitud del vector de límites de intervalos
n_intervalos = len(bucketizer.getSplits())-1
print(f"Salida del bucketizer con {n_intervalos} intervalos")

bucketedData.show()
```

```
Salida del bucketizer con 4 intervalos
```

```
+--------+----------------+
|features|bucketedFeatures|
+--------+----------------+
|  -999.9|             0.0|
|    -0.5|             1.0|
|    -0.3|             1.0|
|     0.0|             2.0|
|     0.2|             2.0|
|   999.9|             3.0|
+--------+----------------+
```

Skip to main content

Cualquier modelo entrenado es un transformador

El resultado de entrenar un modelo sobre un DF es un objeto *Model de la subclase específica del modelo que hayamos ajustado (por ejemplo, `LinearRegressionModel`, `RandomForestClassifierModel` etc), que además es también un `Transformer`, por lo que es capaz de transformar (hacer predicciones) un DF de ejemplos, siempre que tenga el mismo formato (mismos nombres de columnas y tipos de datos) que el DF que se utilizó para entrenar. El motivo es que todas las clases *Model son subclases de la clase `Transformer`, que actúa como una interfaz común a todas ellas, y por tanto, se comportan como transformadores.

- En un modelo entrenado, *transformar* quiere decir añadir una o varias columnas con predicciones al DF resultante.
  - En el caso de los modelos de regresión, añaden una nueva columna, mientras que los de clasificación añaden tres columnas.
- Las predicciones se añaden para cada ejemplo en una nueva columna
- Para facilitar que se mantenga el mismo formato, se suele entrenar un pipeline completo y utilizar su salida (pipeline entrenado) como transformador, como veremos después.

En el ejemplo siguiente, leemos un fichero con un formato peculiar llamado *libvsm*, que es el formato de fichero de datos propuesto por los autores de la primera implementación abierta de las Support Vector Machines (SVM), en lenguaje C++, llamada también **libsvm** y descrita en este enlace. El formato "libsvm" especifica en la primera columna el valor de la target. Después, cada línea del fichero es un ejemplo, y sus features se indican en formato índice:valor donde el índice es un entero con el número de columna, y valor es el valor de esa feature. Sólo se codifican las features con valores distintos de 0, por lo que podemos tener filas como `2.42 1:0.65 4:5.21 7:-4.55`, que indica que las columnas 0, 2, 3, 5 y 6 tienen valor 0 para este ejemplo, y el valor 2.42 es la target.

```
+1 1:0.06992427 2:0.043791715 3:0.010570114 4:0.041858662 5:0.0016906267 6:0.043480627
+1 3:0.0028312805 5:0.0012679701 7:0.010755781
+1 1:0.021851335 2:0.008553068 3:0.0040109805 4:0.032702077 5:0.0016906267 6:0.00776439
+1 2:0.011729923 3:0.009707248 4:0.11212142 5:0.0016906267 7:0.0046096207 9:0.019167876
+1 3:0.005662561 4:0.017441109 5:0.0016906267 6:0.031057592 7:0.006572977 9:0.029816695
+1 1:0.09711705 2:0.018246546 3:0.008808428 4:0.043602776 5:0.0016906267 6:0.015528796
+1 2:0.008210946 3:0.0033975367 4:0.031393997 5:0.0016906267 6:0.018634556 7:0.00645346
+1 2:0.037322484 3:0.0061773392 4:0.028539997 5:0.0016906267 6:0.008470252 7:0.00586678
+1 1:0.06992427 3:0.004530049 5:0.0016906267 7:0.008604625 9:0.08050507 13:0.13395478
+1 2:0.03518977 3:0.008089373 4:0.033636425 5:0.0016906267 6:0.019965595 7:0.0046096207
+1 1:0.037459433 2:0.038122248 3:0.006875967 4:0.022424284 5:0.0016906267 6:0.009982797
+1 1:0.058270227 3:0.009437601 5:0.0016906267 6:0.015528796 7:0.0071705207 9:0.02236252
+1 2:0.02526445 3:0.008711632 5:0.0016906267 6:0.028668547 7:0.0066189426 8:0.12320319
```

Skip to main content

```
from pyspark.ml.regression import LinearRegression
training = spark.read.format("libsvm")\
                     .load("sample_linear_regression_data.txt")
training.show(5, truncate=False)

lr = LinearRegression(maxIter=10, regParam=0.3, elasticNetParam=0.8)  # El objeto a
lrModel = lr.fit(training)   # El método fit de un Estimator devuelve un Transforme

print(f"Clase del objeto almacenado en lrModel: {lrModel.__class__}")

from pyspark.ml import Transformer

print(f"¿Pertenece también a la clase Transformer? {isinstance(lrModel, Transformer

pred = lrModel.transform(training)
pred.show(5, truncate=False)
```

```
+-------------------+--------------------------------------------------------------
|label              |features
+-------------------+--------------------------------------------------------------
|-9.490009878824548 |(10,[0,1,2,3,4,5,6,7,8,9],[0.4551273600657362,0.36644694351969
|0.2577820163584905 |(10,[0,1,2,3,4,5,6,7,8,9],[0.8386555657374337,-0.1270180511534
|-4.438869807456516 |(10,[0,1,2,3,4,5,6,7,8,9],[0.5025608135349202,0.14208069682973
|-19.782762789614537|(10,[0,1,2,3,4,5,6,7,8,9],[-0.0388509668871313,-0.416687005176
|-7.966593841555266 |(10,[0,1,2,3,4,5,6,7,8,9],[-0.06195495876886281,0.654644848029
+-------------------+--------------------------------------------------------------
only showing top 5 rows
```

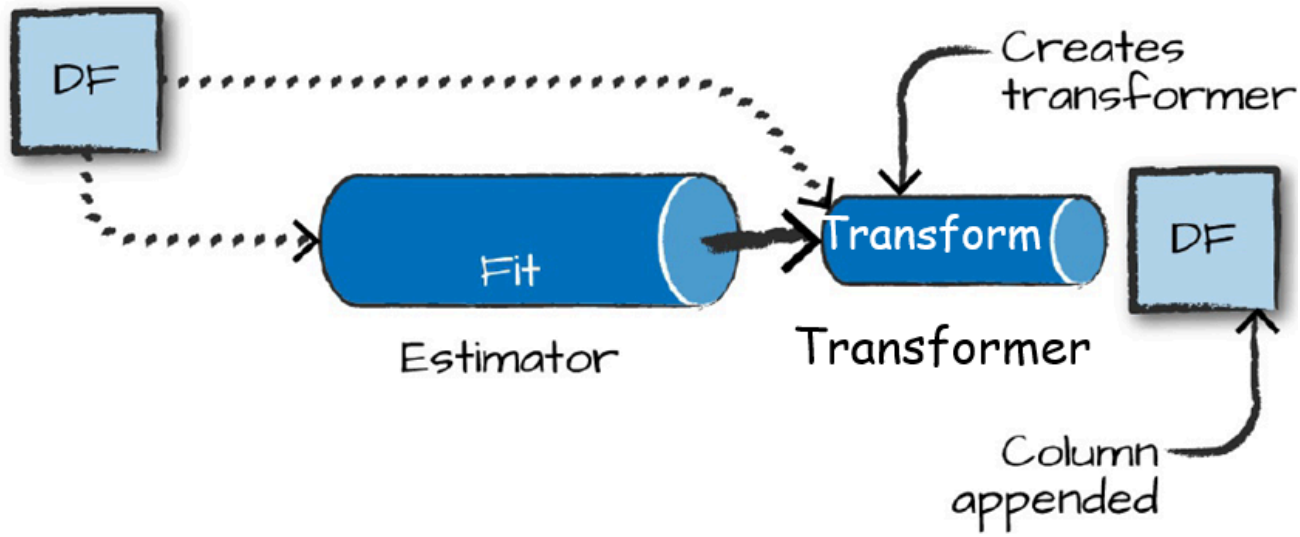
```
Clase del objeto almacenado en lrModel: <class 'pyspark.ml.regression.LinearRegress
¿Pertenece también a la clase Transformer? True
+-------------------+--------------------------------------------------------------
|label              |features
+-------------------+--------------------------------------------------------------
|-9.490009878824548 |(10,[0,1,2,3,4,5,6,7,8,9],[0.4551273600657362,0.36644694351969
|0.2577820163584905 |(10,[0,1,2,3,4,5,6,7,8,9],[0.8386555657374337,-0.1270180511534
|-4.438869807456516 |(10,[0,1,2,3,4,5,6,7,8,9],[0.5025608135349202,0.14208069682973
|-19.782762789614537|(10,[0,1,2,3,4,5,6,7,8,9],[-0.0388509668871313,-0.416687005176
|-7.966593841555266 |(10,[0,1,2,3,4,5,6,7,8,9],[-0.06195495876886281,0.654644848029
+-------------------+--------------------------------------------------------------
only showing top 5 rows
```

Estimadores de Spark ML

Un *Estimator* es un objeto de Spark capaz de realizar transformaciones que primero requieren que ciertos parámetros de la transformación se ajusten (o se aprendan) a partir de los datos. Normalmente requieren una pasada previa (o varias) sobre la columna de datos

Skip to main content

La interfaz Estimator tiene un único método: `fit(df: DataFrame)` que recibe un DataFrame y devuelve un objeto de tipo *Model, el modelo entrenado, que es además un Transformer tal como explicamos anteriormente (*cualquier modelo entrenado es un Transformer*). Una vez entrenado, el modelo resultante puede aplicarse para predecir cualquier conjunto de datos, no solamente los mismos datos con los que fue entrenado.



Es importante notar que Spark llama *modelo* a cualquier cosa que requiera un fit previo, no sólo a los algoritmos de aprendizaje automático que conocemos como modelos propiamente. El motivo es que existen varias piezas de pre-procesamiento que, sin ser algoritmos predictivos, también son **entrenables**. Esto mismo ya ocurría en la biblioteca de `scikit-learn`: por ejemplo la pieza `StandardScaler`, que también existe en la biblioteca de Spark ML puesto que su diseño está muy inspirado por scikit-learn, es una pieza *entrenable*: para estandarizar una variable, primero tenemos que aprender (calcular y memorizar) la media y varianza de la columna que vamos a estandarizar. La estandarización se lleva a cabo restando la media y dividiendo por la varianza, de manera que para la variable X, sus valores estandarizados serán $z_i = \frac{x_i - \mu}{\sigma}$ (a esto se denomina transformación Z-score). A continuación, podemos estandarizar esa columna, pero lo que es más importante, la pieza StandardScaler habrá sido entrenada y tendrá almacenados los valores de μ y σ. Si hemos usado esa pieza para estandarizar los datos con los que después hemos entrenado un modelo, entonces cuando queramos hacer predicciones con ese modelo usando datos nuevos, habrá que estandarizarlos de la misma manera que se estandarizaron los datos de entrenamiento, es decir, restando *la misma media* que habíamos calculado sobre los datos de entrenamiento, y dividiendo por *la misma varianza* de los datos de entrenamiento.

Vamos a ver ejemplos de algunos estimadores comunes.

Skip to main content

StringIndexer

Estimador para pre-procesar variables categóricas. Es el más utilizado. Convierte una columna categórica (da igual el tipo ya que los valores serán entendidos como categorías) en números reales (Double) con la parte decimal a 0 y empezando en 0.0, donde las categorías se representan mediante 0.0, 1.0, 2.0... Entrenar el StringIndexer consiste en generar una pieza, que es el StringIndexer entrenado (clase `StringIndexerModel`), que memoriza internamente la correspondencia entre cada categoría original y su valor codificado. Esa correspondencia ya creada (con los datos de entrenamiento) será aplicada cuando se utilice el `StringIndexerModel` para codificar una variable en datos nuevos para predicción.

La correspondencia creada no es realmente relevante, aunque por defecto Spark asigna un 0.0 a la categoría que más veces se repite, un 1.0 a la siguiente que más veces aparece, y así sucesivamente.

- Además, añade metadatos al DataFrame transformado (resultante de transform) indicando que esa columna es categórica (no es como cualquier otra columna de números reales)
- Los algoritmos que sí soportan variables categóricas (ej: DecisionTree, RandomForest, GradientBoostedTrees) requieren que se las pasemos indexadas
- Los algoritmos que no soportan variables categóricas (LinearRegression, LogisticRegression) requieren el uso de OneHotEncoder (siguiente slide)

IMPORTANTE: al usar un modelo de machine learning para predecir ejemplos nuevos, primero hay que codificar sus variables categóricas siguiendo exactamente la misma codificación que se utilizó con los datos de entrenamiento con los que se entrenó el modelo. Si al crear el dataset de train, se había codificado la categoría “alto” como 1.0, entonces cuando queramos predecir un nuevo dato, antes de pasárselo al modelo, tendremos que codificar “alto” también como 1.0, porque de lo contrario estaríamos falseando (“inventando”) el valor de esa feature del ejemplo que queremos predecir. Por eso cobra utilidad la estructura de Pipeline que comentaremos más adelante.

Es posible utilizar como argumentos inputCols y outputCols, que deben ser dos listas de string de la misma longitud, para codificar un conjunto de columnas (de manera independiente) en un solo artefacto. De esa forma evitamos tener que crear tantos objetos StringIndexer como columnas categóricas queramos codificar: es posible codificarlas todas usando un único StringIndexer. El argumento `handleInvalid` indica qué debe hacer el StringIndexerModel (es decir, ya entrenado) cuando tenga que codificar una categoría que

Skip to main content

podríamos tener un DF más pequeño que el original, lo cual es peligroso), o `keep` (valor recomendado, que hace que se codifique con el primer valor que esté libre, por ejemplo 4.0 si ya teníamos ocupados 0.0, 1.0, 2.0 y 3.0).

```
from pyspark.ml.feature import StringIndexer, StringIndexerModel

df = spark.createDataFrame([(0,"a"), (1,"b"), (2,"c"), (3,"a"), (4,"a"), (5,"c")],

indexer = StringIndexer(inputCol = "category",
                        outputCol = "categoryIndexed",
                        handleInvalid = "keep")

indexerModel = indexer.fit(df)
indexed = indexerModel.transform(df)

# resto de código

# lo uso otra vez
nuevo_df = spark.createDataFrame([("pepe", "c"), ("hola", "c"), ("paco", "b"), ("ma
indexedNuevo = indexerModel.transform(nuevo_df)

print("Resultado de la codificación por defecto (a->0, b->2, c->1):")
indexedNuevo.show()

# otra opción: puedo indicar la correspondencia que deseamos si creamos directament
customIndexerModel = StringIndexerModel.from_labels(inputCol = "category",
                                                    outputCol = "categoryIndexed",
                                                    labels = ["b", "a", "c"]) # cód

print("Resultado de la codificación personalizada (a->1, b->0, c->2):")
customIndexerModel.transform(nuevo_df).show()

print("Entrenamos un único StringIndexer para codificar por separado de una vez amb
otro_indexer = StringIndexer(inputCols = ["name", "category"],
                             outputCols = ["nameIndexed", "categoryIndexed"])

otro_indexer_model = otro_indexer.fit(nuevo_df)
result_df = otro_indexer_model.transform(nuevo_df)
result_df.show()
```

```
Resultado de la codificación por defecto (a->0, b->2, c->1):
```

```
+----+--------+---------------+
|name|category|categoryIndexed|
+----+--------+---------------+
|pepe|       c|            1.0|
|hola|       c|            1.0|
|paco|       b|            2.0|
|mari|       c|            1.0|
| ana|       a|            0.0|
+----+--------+---------------+
```

Skip to main content

```
Resultado de la codificación personalizada (a->1, b->0, c->2):
```

```
+----+--------+---------------+
|name|category|categoryIndexed|
+----+--------+---------------+
|pepe|       c|            2.0|
|hola|       c|            2.0|
|paco|       b|            0.0|
|mari|       c|            2.0|
| ana|       a|            1.0|
+----+--------+---------------+

Entrenamos un único StringIndexer para codificar por separado de una vez ambas colu
```

```
+----+--------+-----------+---------------+
|name|category|nameIndexed|categoryIndexed|
+----+--------+-----------+---------------+
|pepe|       c|        4.0|            0.0|
|hola|       c|        1.0|            0.0|
|paco|       b|        3.0|            2.0|
|mari|       c|        2.0|            0.0|
| ana|       a|        0.0|            1.0|
+----+--------+-----------+---------------+
```

OneHotEncoder

Recibe un conjunto de columnas y convierte cada una (de manera independiente) a un conjunto de variables dummy con codificación one-hot

- Cada variable (con n categorías posibles) da lugar a n-1 columnas (en una sola columna de tipo vector) donde en cada ejemplo, sólo una de ellas tiene valor 1 y el resto son 0 indicando cuál es el valor de la categoría presente en ese ejemplo
- Spark siempre asume que los valores provienen de una indexación previa con StringIndexer: obligatoriamente números reales con la parte decimal a 0
- Entrenar consiste en crear y memorizar la correspondencia entre la categoría original, expresada como número real con la parte decimal a 0, y su vector de 1 y 0. Esa asociación será luego aplicada cuando utilicemos el `OneHotEncoderModel` resultante (one-hot ya entrenado) para codificar nuevos datos siguiendo la codificación que fue aprendida sobre los datos de entrenamiento.

```
from pyspark.ml.feature import OneHotEncoder
from pyspark.ml.linalg import Vectors, VectorUDT  # convertir vectores sparse a der
```

Skip to main content

```
df = spark.createDataFrame([
     (0.0, 1.0, 2.0),
     (1.0, 0.0, 3.0),   # Spark asume que la 3ª col tiene 5 categorías
     (2.0, 1.0, 2.0),   # porque el máximo valor que aparece es 4.0
     (0.0, 2.0, 1.0),   # y asume que vienen de una indexación previa
     (0.0, 1.0, 4.0),   # con StringIndexer y por tanto empiezan en 0.0
     (2.0, 0.0, 4.0)
], ["categoryIndex1", "categoryIndex2", "categoryIndex3"])

encoder = OneHotEncoder(
   inputCols = ["categoryIndex1", "categoryIndex2", "categoryIndex3"],
   outputCols = ["categoryVec1", "categoryVec2", "categoryVec3"]
)
model = encoder.fit(df)
encoded = model.transform(df)

# Lo siguiente simplemente convierte la columna de salida, que eran vectores sparse
# para visualizarlos mejor, aunque lo óptimo es dejarlos como están (como vectores
# en la memoria RAM

toDenseUDF = F.udf(lambda r: Vectors.dense(r), VectorUDT())

encoded.withColumn("categoryVec1", toDenseUDF("categoryVec1"))\
       .withColumn("categoryVec2", toDenseUDF("categoryVec2"))\
       .withColumn("categoryVec3", toDenseUDF("categoryVec3"))\
       .show()
```

```
+--------------+--------------+--------------+------------+------------+-----------
|categoryIndex1|categoryIndex2|categoryIndex3|categoryVec1|categoryVec2|     catego
+--------------+--------------+--------------+------------+------------+-----------
|           0.0|           1.0|           2.0|   [1.0,0.0]|   [0.0,1.0]|[0.0,0.0,1.
|           1.0|           0.0|           3.0|   [0.0,1.0]|   [1.0,0.0]|[0.0,0.0,0.
|           2.0|           1.0|           2.0|   [0.0,0.0]|   [0.0,1.0]|[0.0,0.0,1.
|           0.0|           2.0|           1.0|   [1.0,0.0]|   [0.0,0.0]|[0.0,1.0,0.
|           0.0|           1.0|           4.0|   [1.0,0.0]|   [0.0,1.0]|[0.0,0.0,0.
|           2.0|           0.0|           4.0|   [0.0,0.0]|   [1.0,0.0]|[0.0,0.0,0.
+--------------+--------------+--------------+------------+------------+-----------
```

Pipelines de Spark ML

Es frecuente en Machine Learning extraer características de datos en crudo (“raw”) y prepararlas antes de llamar a un algoritmo de aprendizaje. Sin embargo, puede ser difícil tener control de todos los pasos de pre-procesamiento que hemos hecho al entrenar, para luego replicarlos en otros conjuntos de datos o en el momento de hacer predicciones para nuevos datos con el modelo entrenado. Por ejemplo, para procesar un documento:

- División en palabras

Skip to main content

- Preparación de esas características para el formato que requiere el algoritmo elegido en Spark
- Finalmente, entrenamiento de un modelo.

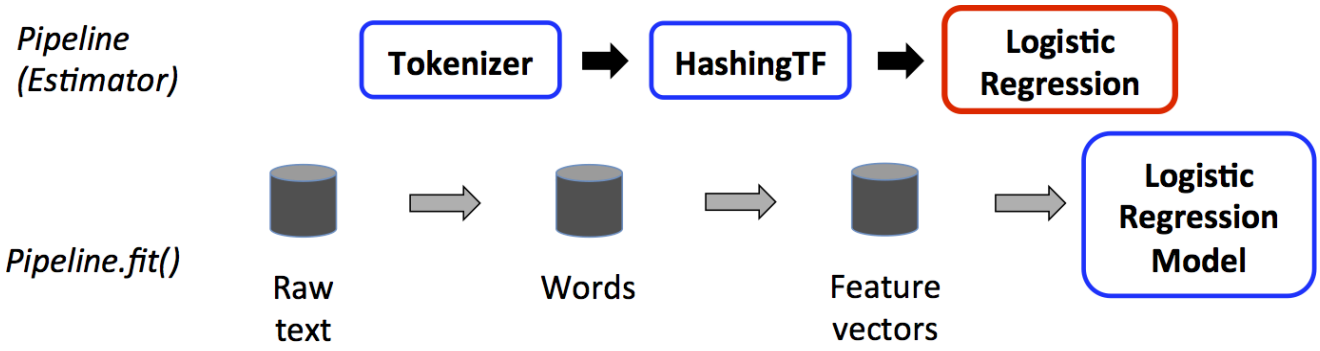
Spark nos proporciona un mecanismo para esto.

> **Note**
>
> Pipeline: secuencia de etapas (PipelineStage, superclase común de Estimator y Transformer) que se ejecutan en un cierto orden. La salida de una etapa es entrada para alguna de las etapas posteriores (no necesariamente la inmediatamente siguiente)

> **Warning**
>
> En general, nunca invocamos directamente a los métodos `fit` o `transform` de ninguna de las etapas que hemos visto. Lo habitual es crear cada etapa para introducirla en un pipeline, y luego invocar al método `fit` del Pipeline, que entrena el pipeline como una sola cosa.

Un Pipeline es un Estimator. El método fit(df) recorre cada etapa, llama a transform() si la etapa es un Transformer, y llama a fit(df) y luego a transform(df) si es un Estimator, pasando siempre el DF tal como esté en ese punto (con las columnas originales más las que le hayan añadido las etapas previas). Es habitual (pero no obligatorio) que la última etapa sea un algoritmo de ML, aunque podría haber varios a lo largo de un pipeline Es importante recordar que un mismo objeto (sea un estimador o un transformador) no puede ser añadido como a dos pipelines diferentes.

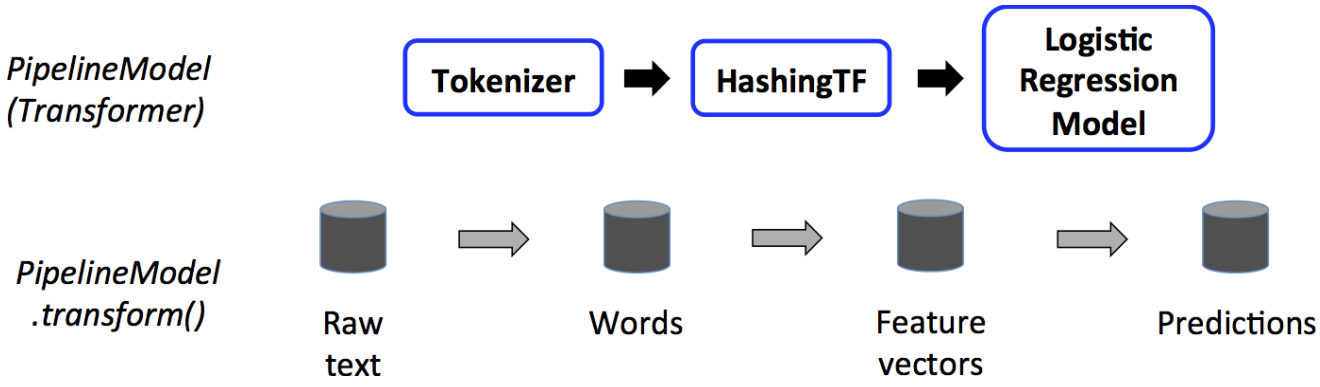
Pipeline antes de llamar a fit:



Las etapas en azul son transformadores mientras que las rojas son estimadores. A continuación vemos el objeto PipelineModel (pipeline ajustado), donde las etapas que eran

Skip to main content

PipelineModel, éste irá llamando a transform para cada etapa, pasando como argumento el DF resultante del transform de la etapa previa.



Lo habitual es llamar a fit sobre el objeto Pipeline una sola vez con los datos de entrenamiento. Esto devuelve un objeto PipelineModel (modelo ajustado), sobre el que podemos llamar a transform tantas veces como queramos sobre conjuntos de datos nuevos (nunca vistos por el modelo, pero contengan las columnas que esperan cada una las etapas) para realizar predicciones sucesivamente.

IMPORTANTE: el orden de las etapas en el pipeline es crucial, ya que probablemente existirán dependencias entre ellas, causadas porque una etapa creará como salida una columna que puede ser necesitada por alguna etapa posterior.

```
from pyspark.ml.feature import StringIndexer, VectorAssembler, Binarizer, VectorSli
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression

trainTest = spark.read.option("header", "true")\
                      .option("inferSchema", "true")\
                      .csv("flights.csv")\
                      .dropna("any", subset=["Month", "DayofMonth", "DayOfWeek", "C
                              "ArrTime", "ActualElapsedTime", "AirTime", "DepDelay
                      .randomSplit([0.8, 0.2], 12345)

trainingData = trainTest[0].cache()
testingData = trainTest[1].cache()

indexers = StringIndexer(inputCols = ["Origin", "Dest", "Month", "DayOfWeek"],
                         outputCols = ["OriginCat", "DestCat", "MonthCat","DayOfWee
                         handleInvalid = "keep") # salida utilizada por el VectorAs

assembler = VectorAssembler().setInputCols(["MonthCat", "DayOfWeekCat", "OriginCat'
                                            "ArrTime", "ActualElapsedTime", "AirTim
                            .setOutputCol("rawFeatures")\
                            .setHandleInvalid("keep") # salida utilizada por el St

scaler = StandardScaler(inputCol="rawFeatures", outputCol="features")\
                        .setWithStd(True).setWithMean(True)    # salida utilizada

binarizerClassifier = Binarizer().setInputCol("ArrDelay")\
```

Skip to main content

```
lr = LogisticRegression().setMaxIter(100)\
                         .setRegParam(0.03)\
                         .setElasticNetParam(0.8)\
                         .setLabelCol("binaryLabel")\
                         .setFeaturesCol("features")

pipeline = Pipeline().setStages([indexers, assembler, scaler, binarizerClassifier,
pipelineModel = pipeline.fit(trainingData)
lrPredictions = pipelineModel.transform(testingData)

lrPredictions.select("rawPrediction", "probability", "prediction", "binaryLabel", '
```

```
+----------------------------------------+----------------------------------------+
|rawPrediction                           |probability                             |
+----------------------------------------+----------------------------------------+
|[2.435588861275205,-2.435588861275205]  |[0.9195011844305776,0.0804988155694224] |
|[2.382940484923568,-2.382940484923568]  |[0.9155171456095604,0.08448285439043957]|
|[2.5408856139784795,-2.5408856139784795]|[0.9269588107889988,0.07304118921100122]|
|[2.067050226813745,-2.067050226813745]  |[0.8876591451126371,0.1123408548873629] |
|[0.5928956889679053,-0.5928956889679053]|[0.6440292750336633,0.35597072496633675]|
+----------------------------------------+----------------------------------------+
only showing top 5 rows
```

El significado de las columnas añadidas por un modelo entrenado es el siguiente.

- En problemas de **regresión**, la única columna que se añade es `prediction`, que se puede configurar en el argumento `predictionCol` del estimador. Es una columna de números reales con las predicciones.
- En problemas de **clasificación**, el modelo entrenado añade 3 columnas al predecir:
  - La columna `rawPrediction`, que se puede configurar al crear el estimador original (antes de entrenar) mediante el argumento `rawPredictionCol`, es una columna de tipo vector, donde el contenido de cada celda es un vector de números reales de tantos elementos como clases tenga el problema, siendo cada elemento una puntuación o score que indica la plausibilidad de que la clase del ejemplo sea la de esa posición. Por ejemplo, el vector `[23.05, 10.22, 6.73]` sería un vector posible para un problema de clasificación de 3 clases. Los scores no tienen que estar en ningún rango fijo, y su significado depende totalmente de qué algoritmo hayamos entrenado. Por ejemplo, si han sido generados al predecir con un `RandomForestClassifierModel`, representan una puntuación que es el voto ponderado de los árboles al predecir, y por tanto, la suma de esos scores será igual al número de árboles (los scores del ejemplo anterior suman 40 y sería compatible con un RandomForest de 40 árboles). En cambio, si los scores han sido generados por un `LogisticRegressionModel` (regresión logística ya entrenada), el significado

Skip to main content

las clases de ese ejemplo, y por tanto, son iguales pero de signo contrario en problemas de clasificación binaria que es donde suele emplearse la regresión logísica.

- La columna `probability`, que puede configurarse al crear el estimador original mediante el argumento `probabilityCol`, es el resultado de normalizar los valores de `rawProbability` en el caso de los modelos que dan un score, como por ejemplo RandomForest. Nótese que normalizar scores **no da como resultado una probabilidad** sino simplemente unos valores que están entre 0 y 1, y que suman 1, pero esto no es suficiente para considerarlos una distribución de probabilidad. Para ver métodos de calcular una verdadera distribución de probabilidad a partir de scores, pueden revisarse los *métodos de calibración* en https://scikit-learn.org/stable/modules/calibration.html. Spark no incorpora métodos de calibración como tal, aunque una regresión logística *adicional* puede bastar si se ajusta bajo las condiciones descritas en esa documentación. Por otro lado, para los métodos que sí ofrecen nativamente como salida una verdadera probabilidad (lo cual no ocurre con casi ninguno, pero sí por construcción con la regresión logística), la columna `probability` es tal cual la probabilidad de pertenecer a la clase positiva.
- La columna `prediction` es la posición (empezando en 0) del máximo valor de la columna `probability`.

Evaluación del modelo

Creamos un objeto **evaluador** del paquete `pyspark.ml.evaluator` que depende del tipo de problema que tengamos (`BinaryClassificationEvaluator`, `MulticlassClassificationEvaluator`, `RegressionEvaluator`, `RakingEvaluator`). El objeto sirve para calcular, sobre cualquier DataFrame, la métrica que hayamos configurado en el argumento `metric`. Debemos indicar cómo se llama la columna del DF que contiene las predicciones (en el caso del evaluador multiclase) o la columna de los scores (en el caso del evaluador de clasificación binaria) y la columna que contiene la target auténtica.

```
# Select (prediction, true label) and compute test error
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Objeto evaluador, para evaluar las predicciones. Compara una columna de predicci
# Hay varias métricas posibles, pero nosotros hemos elegido la más simple: porcenta

evaluator = BinaryClassificationEvaluator(
    labelCol="binaryLabel", rawPredictionCol="rawPrediction", metricName="areaUnder
```

Skip to main content

```
print("Test Error = %g " % (1.0 - auc))
```

```
Test Error = 0.0795499
```

Ajuste de híper-parámetros

Casi todos los algoritmos tienen híper-parámetros cuyos valores debe fijar el usuario antes de iniciar el algoritmo. Un híper-parámetro es un valor que afecta a cómo se lleva a cabo el entrenamiento, pero no varía durante dicho entrenamiento (no es aprendido de los datos, a diferencia de los parámetros del modelo) sino que debe ser fijado al principio. La búsqueda del valor óptimo se hace por ensayo-error de entre un conjunto de valores habituales factibles, aunque existen algunos procedimientos inteligentes, no implementados en Spark, que van guiando la búsqueda.

Spark proporciona herramientas para buscar la mejor combinación de valores de los híper-parámetros, indicándole el conjunto de posibles valores que debe probar con cada uno. Se fija una combinación, se entrena el modelo con un subconjunto de los datos (subconjunto de entrenamiento: el mismo subconjunto para todas las combinaciones que se irán probando) y se evalúa el modelo obtenido con esa combinación utilizando otro subconjunto (subconjunto de validación, que también debe ser el mismo para evaluar todas las combinaciones, con lo que se consigue una evaluación justa).

- Si hay suficientes datos, basta con un solo conjunto de entrenamiento y otro de validación. Se utiliza la pieza `TrainValidationSplit` para buscar automáticamente la mejor combinación de híper-parámetros.
- Si no, se lleva a cabo Cross Validation (validación cruzada). Su misión es asegurar que la métrica es estable y fiable, y para ello, para cada combinación de híper-parámetros que estamos evaluando, no entrena un solo modelo sino *k* modelos independientes. Se particiona el subconjunto de entrenamiento en k partes iguales, disjuntas, llamadas *folds*, y se entrenan *k* modelos independientes, cada uno de ellos utilizando todos los datos del subconjunto de entrenamiento excepto uno de los folds, que se deja fuera para utilizarlo como fold de validación de ese modelo en concreto. Por tanto, cada fold tendrá que haber actuado exactamente una vez como fold de validación de algún modelo. Esto implica que, para cada combinación que estemos evaluando, habremos entrenado *k* modelos, habremos aplicado cada uno de ellos a su fold de validación correspondiente y habremos obtenido *k* valores independientes de la métrica con la

Skip to main content

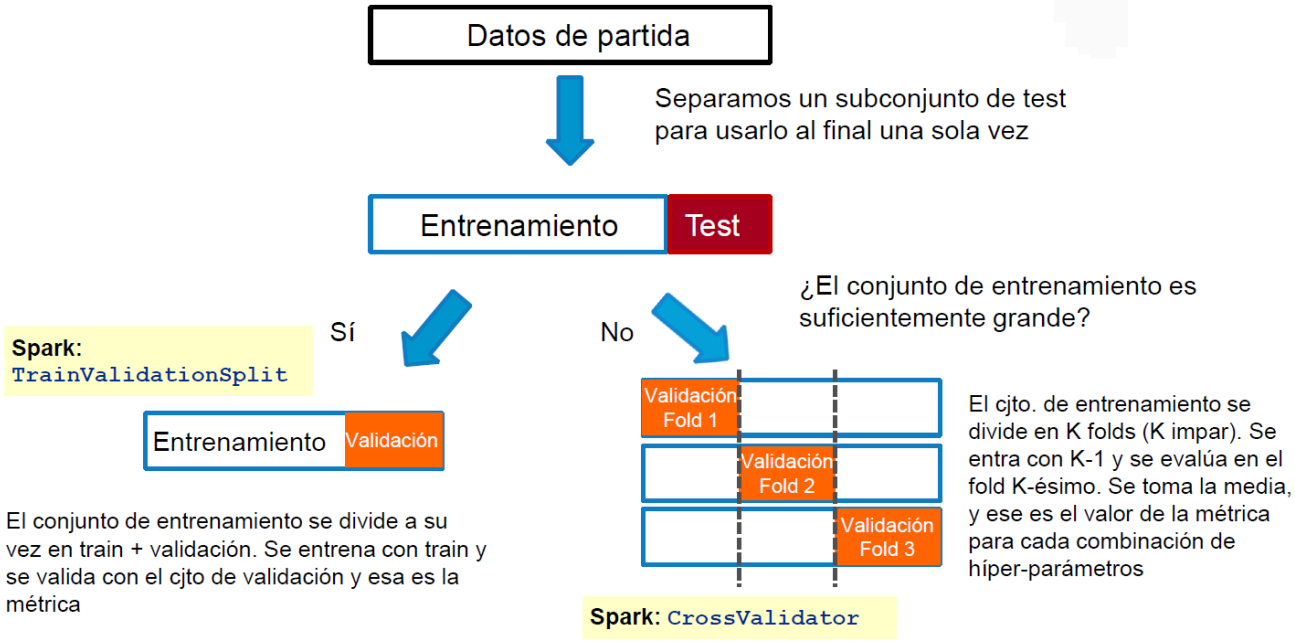
accuracy, precision, recall … , o si el problema es de regresión, RMSE, MAE, …). El **promedio** de esos *k* valores de la métrica es el valor final que asignaremos a esa combinación de híper-parámetros. Como los folds son los mismos para todas las combinaciones de híper-parámetros que estamos evaluando, la comparación es justa. El promedio se considera una métrica más estable y fiable para conjuntos de datos relativamente pequeños que si hiciésemos una sola división de los datos en subconjunto de entrenamiento y de validación y con eso calculásemos una sola vez la métrica. Se utiliza la pieza `CrossValidation` para la búsqueda.

- No siempre es necesario utilizar CrossValidation para el ajuste de híper-parámetros. Si las métricas en el fold de validación en cada caso son muy similares entre sí cada vez que entrenamos distintos modelos con la misma combinación de híper-parámetros, probablemente cualquier fold de validación es suficientemente representativo del conjunto original y podríamos hacer una sola división en entrenamiento/validación, y utilizarla para entrenar un solo modelo con cada combinación de híper-parámetros que queramos medir.

Con cualquiera de las dos herramientas, Spark devuelve el modelo entrenado con la mejor combinación de parámetros, usando para ello todo el dataset (sin dividir en folds ni en entrenamiento/validación). Por tanto, en el caso de utilizar el `TrainValidationSplit`, Spark entrena en total tantos modelos como combinaciones queramos probar, más uno entrenado al final con todo el dataset. En el caso de utilizar el `CrossValidation`, el número de modelos independientes que Spark entrena es igual a *k* por el número de combinaciones de híper-parámetros, más un modelo extra al final, sin división en folds, entrenado con la combinación que ya ha comprobado (gracias a los folds) que funciona mejor.

Antes de empezar todo el proceso, se separa el **subconjunto de test**, que no participará nunca en el proceso. Será utilizado al final, solamente una vez, cuando ya está decidida la mejor combinación de híper-parámetros. El objetivo del subconjunto de test es anticipar cómo se comportará el mejor modelo cuando se aplique datos nunca vistos, lo que permite obtener una métrica que se entregará junto con el modelo entrenado.

Skip to main content



El ajuste de híper-parámetros se realiza sobre objetos `Param` que son variables internas de cualquiera de las piezas que hemos añadido a nuestro pipeline. Cada variable interna con la que configuramos cualquier pieza de pre-procesamiento, sea estimador o sea transformador, es un objeto de la clase `Param` (parámetro) de Spark ML. Esto implica que cualquier variable de cualquier etapa del pipeline puede ser sometida a tuning, no solamente los híper-parámetros de los algoritmos predictivos propiamente.

```
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

# Objeto ParamGrid al que le tenemos que indicar cada uno de los parámetros sobre l
# y la lista de valores posibles que queremos probar con cada uno. Deben ser paráme
# del objeto estimador que se añadió al pipeline (no pueden ser de ningún otro mode
# está almacenado en la variable randomForest que habíamos creado en celdas anteric
paramGrid = ParamGridBuilder()\
    .addGrid(lr.regParam, [0.01, 0.05, 0.1])\
    .addGrid(lr.elasticNetParam, [0.2, 0.8])\
    .build() # como tenemos 2 híper-parámetros con 2 valores posibles cada uno, hay

# CrossValidator es un estimador. Cuando invocamos a fit(), probará todas las combi
# parámetros, invocando con cada combinación al método fit() del objeto pipeline qu
crossval = CrossValidator(estimator=pipeline,
                          estimatorParamMaps=paramGrid,
                          evaluator=evaluator,
                          numFolds=3)

# Como hemos hecho 3 folds, habrá que entrenar 3 veces cada modelo candidato (cada
# y obtener su media de accuracy. En total vamos a entrenar 12 modelos + 1 modelo e
# con todo el dataset de train (sin folds) usando la combinación de híper-parámetrc

cvModel = crossval.fit(trainingData.sample(0.1, seed=123))  # intentamos evitar que
cvModel.bestModel
```

Skip to main content

```
PipelineModel_69b3cb3bb144
```

```
logisticRegressionModel = cvModel.bestModel.stages[-1]
print(f"regParam óptimo: {logisticRegressionModel.getOrDefault('regParam')}")
print(f"elasticNetParam óptimo: {logisticRegressionModel.getOrDefault('regParam')}"
```

```
regParam óptimo: 0.01
elasticNetParam óptimo: 0.01
```